RDPart: A reuse-based OS-level cache-partitioning policy for fairness optimization in cloud data centers

Javier Aznal
Facultad de Informática
Complutense University of Madrid
Madrid, Spain
jaznal@ucm.es

Juan Carlos Saez
Facultad de Informática
Complutense University of Madrid
Madrid, Spain
jcsaezal@ucm.es

Carlos Bilbao
Facultad de Informática
Complutense University of Madrid
Madrid, Spain
cbilbao@ucm.es

Abstract

In cloud data centers, the colocation of multiple services and applications on the same physical server is crucial for maximizing resource utilization and reducing utility costs. Unfortunately, contention for shared resources across cores, such as the Last-Level Cache (LLC), may lead to severe interference among applications. This complicates quality-of-service (QoS) enforcement and causes performance disparities among applications, ultimately degrading user experience and system fairness.

To address these issues, this paper proposes RDPart, an OS-level Reuse-Driven LLC Partitioning policy designed to improve fairness while preserving the QoS of cloud workloads. In contrast to existing fair LLC-partitioning techniques, RDPart eliminates the need for online performance profiling under varying LLC allocations, thereby avoiding undesired QoS violations typically stemming from such invasive exploratory approaches. Furthermore, RDPart adopts a black-box design, making it well-suited for public cloud environments where real-time QoS feedback from applications is unavailable. We implement RDPart in the Linux kernel and evaluate its effectiveness across a diverse set of workloads, including cloud services and HPC/scientific applications. The results reveal that RDPart achieves a 32.1% average fairness improvement with respect to a state-of-the-art black-box cache-partitioning proposal, while keeping QoS violations consistently low across workloads.

CCS Concepts

• **Computer systems organization** → **Multicore architectures**; **Cloud computing**; • **Software and its engineering** → **Operating systems**; • **Hardware** → *On-chip resource management*.

Keywords

Multicore processors, operating system, cache-partitioning, fairness, Linux kernel, Quality of Service, Intel CAT

ACM Reference Format:

Javier Aznal, Juan Carlos Saez, and Carlos Bilbao. 2026. RDPart: A reuse-based OS-level cache-partitioning policy for fairness optimization in cloud data centers. In *Proceedings of the 55th International Conference on Parallel Processing (ICPP '26), September 28-October 01, 2026, Singapore, Singapore.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3832810.3832897

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License.

ICPP '26, Singapore, Singapore

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2657-6/2026/09
https://doi.org/10.1145/3832810.3832897

1 Introduction

The colocation of multiple services and applications on the same physical server (aka multi-tenancy) is a widespread practice for maximizing resource utilization in cloud data centers. Colocation is paramount to increase revenue and reduce utility costs [11, 31, 35, 36], and substantially contributes to a better utilization of the CPU resources per processor socket, which may now feature hundreds of cores in current high-end server products. Despite advances in technology, undesirable effects stemming from shared-resource contention in multicore processors still become apparent under application colocation, making it difficult to deliver quality of service (QoS), fairness or strict performance guarantees. This kind of contention arises as a result of the competition of co-running applications for shared resources among cores, including the Last-Level Cache (LLC), DRAM controller and memory channels [8, 15, 19].

This work focuses on fairness optimization for containerized workloads under QoS constraints. QoS-aware resource-management policies that solely attempt to deliver high throughput may be inherently unfair, causing a number of undesirable effects in the system. For example, equal-priority applications from multiple users could suffer very different performance degradation as a result of sharing the multicore system w.r.t. a situation where each application runs alone on the same system [23, 29, 30]. Severe performance disparities directly translate into bad user experience, leading to potential monetary loss [28]. Our work explores ways to reduce unfairness in multicore server systems, allowing to address these issues.

A popular mechanism to mitigate shared-resource contention effects in the memory hierarchy is to effectively partition the shared LLC. Research on cache-partitioning policies began more than two decades ago, but it has become a hot topic since the adoption of hardware cache-partitioning support in commodity processors [1, 6]. Over the last few years, several resource-management policies leveraging LLC-partitioning have been proposed. They can be generally divided into two categories. The first group of strategies focuses on delivering QoS guarantees mostly for latency-critical (LC) services, while ensuring acceptable system throughput [11, 19, 26]. These policies require feeding the underlying resource manager with actual per-application QoS metrics observed at runtime (e.g., tail latency). This requirement complicates the adoption of these policies in public clouds and virtual environments [15, 31, 35], where applications are usually treated as black boxes not exposing any kind of application-level metrics to the resource manager. Moreover, these QoS-aware policies treat applications other than QoS-constrained services as second-class citizens, making no effort in balancing the slowdown they experience, which inevitably leads to unfairness.

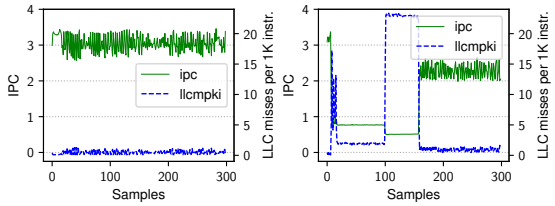


Figure 1: IPC vs. LLCMPKI over time for two SPEC CPU benchmarks. Performance counters are sampled every 500M instructions (coarse interval).

The second category of proposals do not take QoS constraints into consideration, and were primarily designed to optimize fairness [13, 23, 28–30] or throughput [7, 12, 27, 33]. These policies predominantly target long-running CPU-bound applications, and many of them were evaluated only with workloads consisting of single-threaded applications [7, 12, 13, 27, 30, 33]. Crucially, the vast majority of these policies widely rely on observing changes in the number of instructions per cycle (IPC) as a relative performance indicator under contention, and require repeatedly measuring the IPC and other metrics of co-running applications with different LLC allocations at runtime to drive online LLC-partitioning decisions.

These aspects greatly limit the applicability of these techniques in cloud environments for various reasons. First, as we demonstrate in this work, modifying per-application LLC allocations for runtime performance measurement is impractical in the presence of QoS-constrained workloads, as this exploratory approach leads to frequent QoS violations. Second, many cloud workloads exhibit highly irregular CPU-usage patterns over time, and trigger abundant thread activations and deactivations in short time intervals. For these applications, performing efficient and reliable performance measurements with different numbers of LLC ways at runtime (what the aforementioned works do [12, 13, 23, 28, 29]) becomes highly challenging; we elaborate on this issue in Sec. 3. Third, most of these techniques rely on observing changes in the IPC to assess application performance under different LLC occupancies, which may lead to misleading conclusions for many applications [4, 15]. For example, Fig. 1 shows the IPC over time and LLC misses per 1K instructions (LLCMPKI) for two CPU-intensive applications in our Intel-based experimental platform (details in Sec. 6). Although the applications run alone on the platform with a fixed LLC-space allocation (4 cache ways), high fluctuations in the IPC are observed. These fluctuations occur due to program-phase changes (for `astar`), fine-grained variations in the instruction-level parallelism, causing IPC spikes (as is the case for `x264`), etc. All in all, sudden IPC changes may occur due to inherent application behavior, and are not always a result of altering the application's LLC-space allotment.

To address these issues, we propose RDPart, a fair Reuse-Driven LLC Partitioning policy that targets workloads combining long-running CPU-bound HPC-like programs and typical cloud workloads, some of them subject to QoS constraints. Unlike other black-box fairness-oriented approaches [8, 13, 23, 27–29], RDPart supports a wide range of cloud workloads and does not require online application-performance measurements with different LLC allocations to drive partitioning decisions. Instead, it leverages a novel online classification method that allows applications to be divided into different cache-sensitivity classes without altering their current LLC occupancy. To optimize fairness, RDPart (i) identifies applications that are most likely to suffer from severe cache contention, (ii) allocates large LLC partitions to these applications, and (iii) isolates them from other potentially contentious workloads. Crucially, RDPart does not require offline-collected performance information, dynamically reacts to application program phases and operates without any QoS feedback provided by applications, unlike existing QoS-oriented approaches [11, 19, 26]. These aspects, coupled with RDPart's lightweight classification method and black-box nature, make it well-suited for adoption in the OS kernel.

Our work makes the following contributions:

(1) We devised a lightweight mechanism enabling runtime classification of applications into categories based on their LLC sensitivity and contentiousness. This mechanism operates at the thread and the application levels, and continuously monitors key cache-related hardware metrics with performance monitoring counters (PMCs). Unlike most previous work on fairness-oriented cache-partitioning, our monitoring approach is not based on IPC, but instead leverages a novel LLC-reuse metric making it possible to identify applications that are most likely to suffer from severe LLC contention.
(2) Based on the proposed classification mechanism, we designed RDPart, a fairness-optimized strategy that employs a lightweight LLC partition-sharing algorithm.
(3) We implemented RDPart in the Linux kernel. Our open-source implementation [3] relies on a container-aware kernel-level framework [2], which enables RDPart to support a wide spectrum of cloud and HPC multi-threaded and multi-process applications.
(4) We conduct an extensive experimental evaluation of RDPart on a multicore server platform using containerized workloads that combine latency-sensitive services, AI applications, and long-running HPC and scientific programs. The results show that RDPart improves fairness by up to 82% compared to the remaining partitioning strategies considered in our evaluation, and reduces average unfairness by 32.1% w.r.t. a state-of-the-art black-box partitioning approach [15]. In addition to its fairness benefits, which do not impact system throughput, RDPart maintains a consistently low rate of QoS violations across all evaluated workloads.

The remainder of this paper is organized as follows. Sec. 2 introduces the notion of fairness employed in this work as well as the various metrics used. Sec. 3 presents our on-line classification mechanism. The RDPart partitioning strategy is presented in Sec. 4. Sec. 5 discusses related work. Sec. 6 covers the experimental evaluation. Lastly, Sec. 7 concludes the paper.

2 Notion of fairness and metrics

To evaluate the effectiveness of our proposal, we used multi-application workloads. Each workload W consists of N co-running containerized applications ($W = \{a_1, a_2, \cdots, a_N\}$). In this work, we adopt a notion of fairness commonly used in previous work [13, 28, 30]: a policy is deemed to be fair if equal-priority applications in a workload are subject to a similar degree of performance degradation resulting from sharing the platform.

To quantify the relative performance degradation of each application a_i in the workload, we use the *Slowdown* metric [8, 14, 30]. Specifically, let $Perf_{alone,a_i}$ be the performance of a_i observed when it runs alone on the system, and let $Perf_{res,a_i}$ be a_i's performance when running together with the other applications in W under a specific resource management policy. $Slowdown_{a_i}$ is defined as $Perf_{alone,a_i}/Perf_{res,a_i}$. To quantify the performance of a latency-sensitive application, we use its achieved request per second (RPS) rate[1] in the two scenarios (i.e., running alone and together with the other applications, respectively). For batch applications, we measure performance with the inverse of the completion time observed in the two aforementioned scenarios.

To measure the degree of fairness, we use the lower-is-better Unfairness metric [8, 23, 30], defined as the ratio of the standard deviation of slowdowns ($\sigma_{Slowdown}$) to their arithmetic average ($\mu_{Slowdown}$):

$$Unfairness = \frac{\sigma_{Slowdown}}{\mu_{Slowdown}} \quad (1)$$

Apart from unfairness reductions, we also report the system throughput achieved when running each workload. To this end, we employ the widely used higher-is-better STP metric [8, 14, 20], also referred to as *Weighted Speedup* and expressed in terms of the slowdown of each application as follows:

$$STP = \sum_{i=1}^{N} \frac{Perf_{res,a_i}}{Perf_{alone,a_i}} = \sum_{i=1}^{N} \frac{1}{Slowdown_{a_i}} \quad (2)$$

3 Online application classification

Our proposal leverages PMCs to classify applications at runtime into different categories based on the LLC-access behavior of their individual threads. The classification method operates at two levels: per-thread and per-application. The class of individual application threads is determined via continuous monitoring, and the application class is determined based on the dominant application-wide behavior. The remainder of this section describes the set of application classes used (Sec. 3.1), the per-thread classification method (Sec. 3.2) and per-application classification strategy (Sec. 3.3)

3.1 Application classes

In this work, we adopt the application categories proposed in [13, 29], which are based on an application's degree of LLC sensitivity and contentiousness. Particularly, three application classes are considered: *cache sensitive*, *streaming* and *light sharing*. Fig. 2 illustrates the differences between classes by showing the slowdown and per-thread average LLC misses per 1K cycles (LLCMPKC) for five multithreaded applications running in a 4-core container with different LLC ways in our experimental platform. Applications in the cache-sensitive category include programs whose performance is substantially degraded under a low LLC occupancy, like `in-memory-analytics` and `LU`. Failing to fulfill the cache-occupancy demands of this type of application may cause a large slowdown, thus jeopardizing system-wide fairness. The streaming category encompasses bandwidth-intensive cache-insensitive applications that pollute the LLC, while not extracting much performance benefit from extra space in the LLC. The `deepspeech`

[1]As described in Sec. 6, we generate client requests so as to reach a maximum target RPS, which may not be enforced under contention in the multi-application setting.

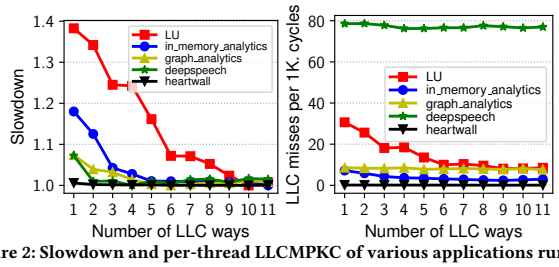


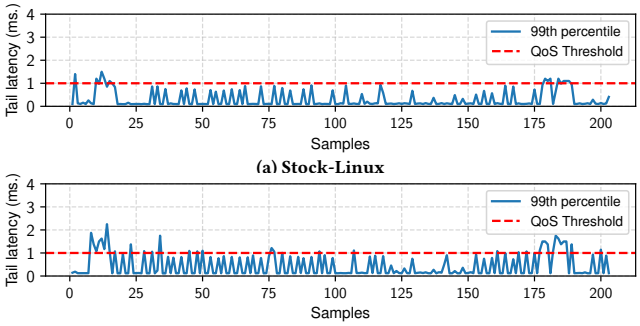
Figure 2: Slowdown and per-thread LLCMPKC of various applications running on a 4-core container with different LLC-way allocations

and `graph-analytics` applications fall in this streaming category; these applications typically exhibit a high LLCMPKC despite increasing their LLC-way allocation. Lastly, the light-sharing class is devoted to applications that are not contentious to others and exhibit largely LLC-insensitive behavior, like `heartwall`. The working set of this type of applications typically fits in the per-core private cache levels, giving rise to a very low LLC request rate.

3.2 Per-thread classification

Motivation. While separating light-sharing threads from the other two categories is straightforward by examining the LLC-request rate, distinguishing streaming from cache-sensitive threads is challenging, as both may exhibit high LLC access and miss rates. The differences between thread types become clearly apparent only when examining the relative performance benefit with different cache-way allocations [12, 23, 29]. To assess an application's degree of cache sensitivity, previous work on cache-partitioning leveraged online profiling during the execution of a multi-program workload [12, 13, 29]. The overall idea of this profiling method is to continuously measure the performance of a selected application with PMCs under different cache allocations, while the remaining applications in the workload are assigned to a separate complementary LLC partition. This facilitates the identification of streaming and cache-sensitive threads.

Notably, this profiling approach is subject to critical issues making it impractical for cloud-like workloads. For example, during a profiling stage, a suboptimal LLC-partitioning mode is imposed while observing the performance of a single application with different cache sizes, and the rest of the programs are confined in a separate partition as big as the remaining LLC size. This exploration method is potentially harmful for latency-constrained (LC) workloads, which may incur additional QoS violations that would not normally occur when the LLC is shared among applications. Fig. 3 demonstrates this issue for a multiprogram workload including the popular `memcached` application [25]. We tracked `memcached`'s tail latency in two scenarios: under Stock-Linux, which does not partition the LLC (Fig. 3a), and under an artificial resource management policy that periodically alters per-application LLC allocations to mimic the aforementioned profiling approach [12, 13, 29] but without measuring application performance (Fig. 3b); while profiling mode is not active, the LLC is not partitioned. As the results reveal, altering the LLC allocation for profiling purposes introduces additional spikes in the tail latency, causing extra QoS violations. So, altering the LLC allocation just to observe the impact on application performance –what many proposals do [12, 13, 23, 28, 29]– is harmful from the QoS standpoint.



(a) Stock-Linux

(b) Changing LLC-space assignment

Figure 3: memcached's tail latency observed every second while running in a multiprogram workload. The QoS threshold is typically set to 1ms in this popular cloud benchmark [25].

benchmark	changes/sec.
EP	1.20
LU	2.36
BLAST	3.46
leukocyte	134.42
graph_analytics	1123.21
deepspeech	1474.73
in_memory_analytics	4547.36
memcached	120368.23

Table 1: Average changes in the runnable thread count per second for 4 HPC/scientific programs and 4 cloud applications (in italics).

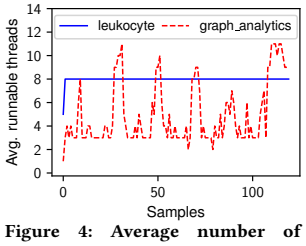
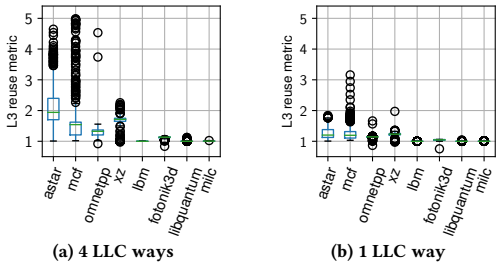


Figure 4: Average number of runnable threads observed every 500ms for an HPC program and a cloud application.

We should also highlight that this type of profiling technique also relies on the assumption that the set of application threads being profiled remains runnable for some time (at least several tens of milliseconds) without blocking; this is crucial for the performance measurement to be effectively performed with different LLC-way allocations. In the aforementioned works, the proposed policies were evaluated with long running, highly CPU-bound applications only, where this assumption holds. Regrettably, this is not the general case in cloud-like workloads, where threads may block very often. Table 1 shows the average number of times per second that threads become runnable or block in four HPC/scientific programs and four cloud workloads running in an 8-core container. While threads of these CPU-intensive HPC applications rarely block, cloud-like workloads trigger abundant thread activations/deactivations per second, with thread-state changes that can be several orders of magnitude more frequent than for HPC applications. The case of `memcached` is particularly revealing with activations/deactivations occurring every few microseconds. Therefore, the said assumption is invalid for many cloud-like workloads. Moreover, as Fig. 4 shows, the degree of CPU utilization tends to vary more significantly over time in cloud workloads than in HPC/scientific workloads, even under coarser observation intervals.

Our approach. To overcome these shortcomings, our proposal relies on continuous per-thread monitoring and performs application classification without altering the cache allocation imposed by the underlying LLC-partitioning algorithm. A key feature of our classification technique is its reliance on a per-thread *LLC reuse metric*. We define this metric as the ratio of the number of cache lines loaded in L2 to the number of LLC misses observed during a given



(a) 4 LLC ways

(b) 1 LLC way

Figure 5: LLC reuse metric distribution for four cache-sensitive applications and four streaming programs running with 4 cache ways and 1 cache way.

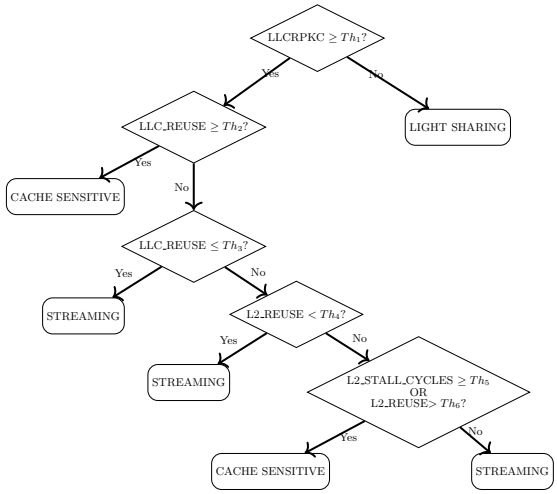


Figure 6: Decision tree for the per-thread classification method

monitoring interval. The greater the ratio, the more cache lines in the LLC are being reused. This reuse metric allows us to separate cache-sensitive from streaming-like threads without exploring different cache sizes. Fig. 5a shows the distribution of the LLC reuse values gathered over time with PMCs for 8 single-threaded applications running alone with four LLC ways on our platform. Although these applications exhibit different program phases, their behavior mostly falls in the cache-sensitive (4 applications on the left) or streaming categories (remaining applications on the right). As it is evident, cache-sensitive applications exhibit larger median values for the reuse metric (greater than 1.2), whereas for streaming applications the reuse metric values orbit around 1.

While the LLC reuse metric plays a crucial role in our proposed online classification, it is not enough by itself to differentiate streaming from cache-sensitive programs. The data gathered on our platform reveals that a thread can be definitely identified as cache sensitive for metric values greater than 1.28; streaming behavior is clearly observed for metric values no greater than 1.11.[2] However, values in the (1.11,1.28) interval require the utilization of further metrics to aid in the classification procedure. To identify these metrics and the corresponding thresholds for identification, we used PMC data gathered for various programs to automatically build a decision-tree-based model like the one depicted in Fig. 6. The model

[2]To corroborate this trend, we gathered PMC samples for fixed instruction windows with different cache ways and built curves similar to those of Fig. 2

Table 2: Mapping of Intel Skylake-X PMU events to functionally similar events available on AMD EPYC processors based on the Zen 5 microarchitecture

Intel Skylake-X event	Zen 5 event	Event select / unit mask	Observations
INST_RETIRED.ANY	PMCx0C0 [Retired Instructions]	0xC0 / 0x00	Direct equivalent
CPU_CLK_UNHALTED.THREAD	PMCx076 [Cycles Not in Halt]	0x76 / 0x00	Direct equivalent
LONGEST_LAT_CACHE.REFERENCES	L3PMCx04 [L3 tag lookup state]	0x04 / 0xFF	Closest Zen 5 event. It is collected using AMD's shared L3 PMU, and requires per-core filtering for thread-specific measurements.
LONGEST_LAT_CACHE.MISS	L3PMCx04 [L3 tag lookup state]	0x04 / 0x01	Same observations as in the previous event.
L2_LINES_IN.ALL	PMCx165 [L2 Fill Response Source]	0x165 / 0xFE	Closest Zen 5 L2-fill signal. It counts L2 fill responses according to their source.
L1D.REPLACEMENT	PMCx044 [Any Data Cache Fills by Data Source]	0x44 / 0x5F	Closest L1D line-allocation signal. AMD counts data-cache fills rather than replacements.
CYCLE_ACTIVITY.STALLS_L2_MISS	PMCx0D6 [Cycles with no retire]	0xD6 / 0xA2	Closest Zen 5 load-delay proxy. It counts cycles with no retirement while the oldest operation is waiting for load data. Unlike the Intel event, it is not restricted to L2 misses and measures no-retirement cycles.

employs 3 additional metrics: the number of LLC requests per 1K instructions (LLCRPKC), the percentage of pipeline stall cycles due to L2 misses (L2_STALL_CYCLES), and a L2 reuse metric (i.e., ratio of the number of L1 cache-line fills to the number of L2 misses). While the LLCRPKC metric is used to separate light-sharing threads from the remaining ones, the other two metrics allow us to properly identify cache-sensitive and streaming threads for reuse metric values in the aforementioned conflicting interval. The per-metric thresholds in the decision tree (Th_i) were obtained automatically when building the model using a CART (Classification and Regression Trees) algorithm. The threshold values specific for our platform can be found in RDPart's source code [3]. To build the model, we used PMC data gathered for a total of 20 single-threaded benchmarks from the SPEC CPU 2006 and 2017 suites. For each benchmark, we sampled PMCs every 50M instructions for the whole execution of each program with 2 and 4 cache ways. Note that, for the evaluation of our proposal, we used a broader set of applications; 24 of these were not used to build the model.

A preliminary evaluation of the effectiveness of the aforementioned classification model revealed that class predictions were accurate for application LLC allocations above 3.75MiB (equivalent to assigning 1.5 cache ways on our platform). Unfortunately, frequent class mispredictions became apparent for a smaller occupancy. We found that this was caused by the lower cache-line reuse opportunities in modest LLC-space allocations. Particularly, the fewer LLC lines are allotted, the more likely a line present in the LLC is to be replaced. As shown in Fig. 5b, assigning a single LLC way to the application (occupancy limited to 2.5MiB) severely impacts the LLC reuse metric, which reaches smaller median values than those of Fig. 5a for cache-sensitive applications. As a result, LLC reuse values for streaming and cache-sensitive programs become much closer, causing mispredictions with the aforementioned classification model. To address this issue, we built another decision-tree-based model with the same metrics but using data gathered with a single cache way. When the LLC occupancy falls below 3.75MiB, RDPart uses this alternative model to drive class predictions.

Although our proposal was evaluated using an Intel Xeon "Skylake" processor (see Sec. 6 for details of the experimental platform), its reliance on commonly available performance events facilitates its implementation on other x86 server processors. To illustrate the portability potential of RDPart's per-thread classification method, Table 2 presents a set of PMC events supported by AMD Zen 5 EPYC processors that are functionally equivalent to those used in our Intel Skylake implementation, according to the respective PMU documentation [5, 16]. As indicated in the table's Observations column, the last three Intel events do not have exact Zen 5 counterparts; however, Zen 5's PMU provides suitable proxy events with closely related semantics. Despite using similar PMC events, the decision-tree model must be retrained using data from the target platform to determine the appropriate thresholds for that platform.

3.3 Per-application classification

The goal of the per-application classification method is to determine the overall class of every active application based on the observed behavior of their threads during a certain *monitoring period*. The length of this period is established via a configurable parameter, whose default value is 500ms. The per-application classification method relies on the per-thread model described earlier and was carefully designed to fully support cloud-like workloads.

RDPart associates three per-class counters with each application, to keep track of the number of streaming, cache-sensitive and light-sharing per-thread classification samples registered during the monitoring period. PMCs are sampled every 50M instructions on a per-thread basis –a suitable instruction window for cloud workloads, where threads exhibit short CPU bursts. When a thread completes that instruction window, a PMC-overflow interrupt is

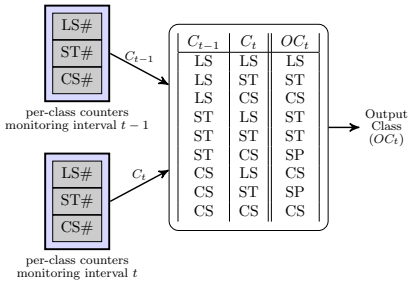


Figure 7: Class smoothing filter

raised, PMC values are gathered, and the corresponding performance metrics required for the thread-level classification are calculated. These metrics are used as input to the decision-tree-based model described in the previous section, which provides a class prediction for that thread. The per-application counter associated with that class is incremented. At the end of the monitoring period, the application-wide class is determined based on the aforementioned counters; specifically, the selected class is the one with the largest associated class-specific counter. These counters are then reset in preparation for a new monitoring period. We should highlight that RDPart's classification method allows the same thread to increment class-specific counters multiple times during a single monitoring interval. This feature is deliberate, as it allows threads with a more CPU-intensive profile to be granted a higher weight when determining the application-wide class.

We observed that frequent application class changes may cause RDPart to alter the underlying LLC partitions often, thus introducing potential overheads. Enlarging the monitoring period reduces the impact of those undesired changes, but limits RDPart's ability to quickly react to applications entering/exiting the system (or blocking) in the middle of the period, which could degrade performance and QoS. (Note that RDPart initially maps newly created applications to an LLC-partition that spans the entire LLC, like other proposals [15].) To address this problem, without altering the period duration, RDPart leverages a class-smoothing filter, which suppresses sudden class changes in consecutive monitoring intervals. The filter, depicted in Fig. 7, is applied after determining the dominant application-wide class (C_t) based on the class-specific counters. It also takes into consideration the application-wide class obtained in the previous monitoring interval (C_{t-1}), and produces an output class (OC_t) that is used as input to RDPart's cache-partitioning algorithm. This output class takes one of the following values: ST (Streaming), CS (Cache Sensitive), LS (Light Sharing) or SP (Special). The *special* class –not considered in the per-thread classification– is reserved for applications that could be oscillating between the ST and CS states, which deserve special treatment for QoS reasons; we will elaborate on this aspect in Sec. 4.

We tested the effectiveness of the filter via extensive experiments, but were unable to discuss the full set of results in the paper due to space constraints. To illustrate the filter's efficacy in a compact way, Figure 8 shows the results for two applications used in our experiments (`xz_s` and `DeepSpeech`), which were not used to build the per-thread classification model. In particular, Figs. 8a and 8c display the raw class obtained over time, whereas Figs. 8b and 8d depict the filtered class (for simplicity, SP samples are displayed as ST here). As it is evident, the filter allows us to better capture

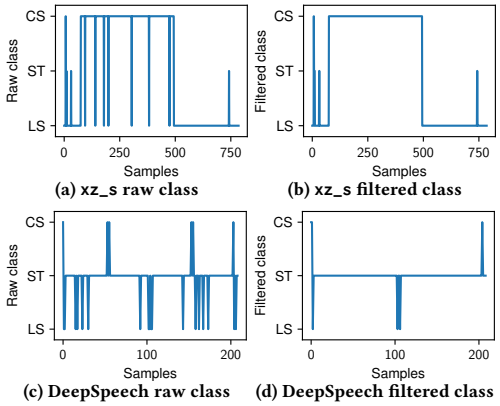


Figure 8: Effects of the class filter on the xz (a) and DeepSpeech (b) applications

the dominant cache-sensitive and streaming behavior of `xz_s` and `DeepSpeech`, respectively. The vast majority of sudden class transitions are successfully filtered out throughout the execution.

4 Cache-partitioning algorithm

RDPart's partitioning algorithm is executed at the end of the *monitoring period*, immediatly after gathering the class of each application. It has been designed to maximize system-wide fairness, and, to this end, it follows two main principles. First, it prioritizes allocating as much LLC space as possible to cache-sensitive applications (CSAs), which are particularly subject to high slowdowns under LLC contention. By granting them larger LLC allocations, the algorithm reduces their slowdown, thereby helping to keep system-wide unfairness under control. Second, it confines streaming programs in small partitions that do not overlap with those associated with CSAs. This effectively isolates CSAs from streaming programs (which cause high cache pollution), and "wastes" only a reduced portion of the LLC in these applications. Note that these principles are inspired by the LFOC [13] and LFOC+ [29] partitioning strategies. Crucially, unlike RDPart, none of these approaches supports cloud-like workloads, or takes QoS-constraints into consideration. Moreover, they systematically degrade the performance of streaming applications. To address this issue, RDPart treats LC streaming applications separately from batch streaming workloads.

RDPart's partitioning algorithm leverages partition-sharing [10, 14]. Instead of assigning separate cache partitions to each application (aka strict cache-partitioning), it groups applications into clusters, and associates a cache partition (set of LLC ways) with that cluster. LLC partition-sharing becomes more effective than strict LLC partitioning on platforms with coarse-grained partitioning support, like most current commercial server processors [29]. Our proposed partitioning strategy –outlined in Algorithm 1– accepts as input the sets of streaming, light-sharing, cache-sensitive and special applications identified by our classification method. Before creating the various cache clusters (partitions), it traverses the list of streaming applications, and moves those that are also latency sensitive to the *special* class, to give them preferential treatment over regular streaming workloads. Notably, LC applications in the remaining classes are treated equally to non-LC ones.

The first step of the algorithm reserves as many "small" cache-partitions as the number of special applications; the size of these

Algorithm 1 RDPart's cache-partitioning algorithm

```
1:  Input: ST, CS, SP and LS are the sets of streaming, cache-
    sensitive, special, and light-sharing applications, respectively;
    ways_small is a configurable parameter, and llc_ways is the
    number of ways of the LLC.
2:  function RDpartitioning(ST, CS, SP, LS, llc_ways)
3:      < avail_ways, Clusters>  ← <llc_ways, ∅>;
    ▷ Step 1: Create clusters for special applications
4:      for each st_app ∈ ST do
5:          if st_app is latency sensitive then
6:              move application from the ST set to the SP set
7:      for each special_app ∈ SP do
8:          Create a cluster S consisting of ways_small ways;
9:          Map special_app to S, and add S to Clusters ;
10:         avail_ways ← avail_ways − ways_small;
    ▷ Step 2: Create cluster for apps in LS
11:     if |LS| ≠ 0 then
12:         Create a cluster L consisting of llc_ways ways;
13:         Add L to Clusters ;
14:         for each ls_app ∈ LS do
15:             Map ls_app to L ;
    ▷ Corner case: No cache-sensitive applications
16:     if |CS| == 0 then
17:         Create a single cluster U consisting of avail_ways;
18:         Add U to Clusters;
19:         Map all applications in ST to U;
20:         return Clusters
    ▷ Step 3: Create as many streaming clusters as needed
21:     < ways_str, STC>  ← allocStrClusters(ST);
22:     avail_ways ← avail_ways − ways_str;
23:     Clusters ← Clusters ∪ STC;
    ▷ Step 4: Distribute remaining space among apps in CS
24:     CSC  ← allocSensCluster(CS, avail_ways);
25:     Clusters ← Clusters ∪ CSC;
26:     return Clusters
```

partitions is given by the *ways_small* parameter whose default value is 2 LLC ways[3]. Step 2 creates an additional cache-cluster encompassing the full LLC, which accommodates all light-sharing applications. This category of programs makes scarce LLC requests, so RDPart does not limit the number of cache ways they can use.

In Step 3, a set of clusters of *ways_small* LLC ways each, is created to accommodate streaming programs by invoking `allocStrClusters()` (pseudo-code omitted due to space constraints). This procedure takes into account per-application bandwidth consumption measurements gathered during the monitoring interval. It begins by allocating a single cluster to hold as many streaming applications as possible without exceeding a maximum allowed aggregate memory bandwidth in the partition. If that limit is exceeded, additional clusters are allocated as needed. Packing multiple streaming programs in the same partition allows to makes more room in the LLC for the remaining applications, which use the cache more effectively. Moreover, limiting the number of streaming applications in a partition based on their bandwidth consumption reduces bandwidth contention, which may become apparent when isolating multiple streaming applications in small LLC partitions [29].

Step 4 distributes the remaining LLC space among cache-sensitive applications (CSAs) by invoking `allocSensCluster()`. The base RDPart implementation allocates a single partition with the remaining LLC space (`avail_ways`) and shares it among all CSAs. This design decision relies on the observation that mapping multiple CSAs to the same partition is beneficial for system-wide performance and fairness in many cases [29]. For comparison purposes, we also experiment with an RDPart variant –referred to as RDPart-E– that evenly distributes the remaining space among CSAs, which are assigned to separate LLC partitions. Our results in Sec. 6 reveal that the base RDPart version achieves a higher average reduction in unfairness and slightly better system throughput than RDPart-E.

Implementation Notes. We implemented RDPart in the Linux kernel v6.2.16, on top of the open-source PMCSched framework [9]. Among other things, PMCSched allows to create custom resource management strategies as *plugins* within a loadable kernel module. RDPart's plugin heavily relies on PMCSched's new support to seamlessly manage containerized applications at the kernel level [2]. Although this work evaluates RDPart's effectiveness with containerized workloads, its implementation could be extended to support KVM-based virtualized workloads; in Linux KVM, VMs are exposed to the OS kernel as multi-threaded processes [15], and their virtual CPUs (vCPUs) are seen as threads by the kernel.

Notably, on systems featuring clustered (chiplet-based) processor designs and/or multiple CPU sockets, our implementation runs multiple independent RDPart instances, each operating on a separate LLC (L3 cache) –shared by separate subsets of cores in the platform. Each RDPart instance is in charge of partitioning a specific LLC. This per-LLC implementation avoids centralized coordination, and scales more effectively as the number of LLCs increases.

5 Related work

Research on cache-partitioning policies has rekindled a lot of attention in the last decade [7, 8, 11–13, 15, 19, 21, 27–29]. The LLC can be partitioned via hardware support [1, 6] or by employing software mechanisms, such as page coloring [21, 34]. Our proposed OS-level strategy relies on hardware way-partitioning support.

Our proposal constitutes a black-box partitioning policy: it does not require being fed with any kind of application-level metric (e.g., tail latency, frames processed per second, etc.) to function. Many other black-box policies have been proposed in previous work [7, 12, 21, 27, 33], some of them with the primary goal of fairness optimization [13, 23, 28–30]. The vast majority of black-box fairness-oriented policies were designed and evaluated for highly CPU-intensive workloads, consisting of applications where threads remain active for long time periods [8, 13, 23, 28–30]; this assumption is crucial for the underlying online performance monitoring method they all leverage. None of these policies care about QoS-constrained workloads, and rely on measuring application performance with different LLC occupancies to drive cache-partitioning decisions. As we showed in Sec. 3.2, adjustments in the LLC-space distribution just for profiling purposes cause QoS violations. Unlike previous approaches, our proposal does not require these invasive exploratory procedures –thanks to its lightweight application classification method–, does fully support a wide range of cloud workloads (even with irregular CPU usage profiles), and does not rely on the IPC metric for application characterization, thus avoiding misleading conclusions from LLC-unrelated IPC changes.

[3]Using LLC partitions smaller than two cache ways has been proven to be harmful for streaming-like applications [29].

In this work, we compare the effectiveness of RDPart with that of CacheMan [15], a state-of-the-art black-box fairness-oriented LLC-partitioning strategy. Like RDPart, CacheMan eliminates the need for exploratory performance measurements with different LLC allocations, and its lightweight design (also free from floating-point calculations) makes it suitable for adoption in the OS kernel or VMM. In fact, a kernel-level implementation of CacheMan was considered for its experimental evaluation [15]. CacheMan defines a set of overlapping cache partitions, all with different sizes; the lower the partition ID (level), the larger the partition. To deliver fairness to multiple applications/VMs, it continuously monitors the per-application LLC occupancy and gradually moves applications across different partition levels, so as to match their target *fair* LLC occupancy – proportional to the number of cores assigned to the application. Applications whose occupancy falls below their fair allocation are promoted to upper partitioning levels (larger partitions), whereas those with largely unfair LLC occupancies are relegated to smaller partitions. Our experiments reveal that RDPart provides superior fairness compared to CacheMan across a wide range of multi-program workloads. This is mainly because CacheMan operates regardless of an application's degree of cache sensitivity, which is crucial for fairness-optimized LLC-partitioning.

Most of the previously proposed policies that explicitly deal with QoS-constrained services require being fed with application-level QoS metrics [11, 19, 26]. This requirement impedes widespread adoption of these policies in public clouds [15, 31, 35], where the underlying resource manager treats applications as black boxes that do not expose any kind of application-level metrics at runtime. Although these techniques deal with other sources of contention in the platform beyond the LLC (e.g., disk or network bandwidth), they are inherently unfair to applications other than QoS-constrained services, as they make no effort in balancing the slowdowns they experience. Our work focuses instead on the design of a black-box partitioning policy that delivers system-wide fairness, while attempting to minimize QoS violations of LC services.

6 Experimental evaluation

In this section, we begin by describing our experimental setup and introducing the applications and methodology employed in our experiments (Sec. 6.1). The detailed discussion of the results can be found in Sec. 6.2.

6.1 Experimental setup and methodology

To assess the effectiveness of RDPart, we employ containerized workloads that combine cloud applications with HPC and scientific programs. For the experiments we use a dual-socket server system with 96GB DRAM that integrates two 20-core Intel Xeon Gold 6138 (Skylake) processors. Each processor has an 11-way 27.5MB LLC (L3) with way-partitioning support [1] shared by all cores. All cores run at 2Ghz and have two private cache levels (64KB L1 + 1MB L2). Many of the applications in the workloads are batch programs, whereas others are latency-critical (LC) services. In our experiments we devote one socket of the machine (CPUs 20-39) to run the clients generating requests to LC services; the other socket (CPUs 0-19) runs the multi-container workload, where fairness and QoS are assessed. Containerized applications and clients are pinned to their respective sockets, with data and code allocated to the memory banks associated with their corresponding socket. This separation minimizes interference, enabling a clean evaluation of our proposal's raw potential.

Fig. 9 shows the composition of the 40 randomly generated workloads used in our experiments, which combine varying amounts of streaming, cache-sensitive and light-sharing applications. For the sake of clarity, applications that predominantly fall in the cache-sensitive category are denoted in **bold** in the corresponding labels of Fig. 9, whereas streaming programs are denoted in *italics*. Each workload includes five or six applications. Particularly, each 6-application mix (workloads 1-22) combines 4 single-threaded benchmarks from SPEC CPU –each running in a single-core container– and 2 multi-threaded or multi-process applications –each one assigned to an 8-core container. By contrast, workloads 23-40 include five multi-threaded or multi-process applications, each running in a 4-core container. Thus, the set of containers in each workload utilizes all available cores in the socket.

Workloads in Fig. 9 include 40 different applications, eight of which are cloud benchmarks. Particularly, we experimented with 3 cache-sensitive benchmarks from Cloudsuite [24]: `data-caching` (LC benchmark based on memcached), `graph-analytics` (based on the Apache GraphX library) and `in-memory-analytics` (based on Apache Spark's MLlib). We also tested with 3 LC benchmarks from TailBench [17] –`SPECJbb`, `imgDNN` and `xapian`–, and with two additional cloud benchmarks –`DeepSpeech` and `imgViT`. The machine-learning `DeepSpeech` [22] benchmark converts a three-minute audio recording into text by leveraging Mozilla's DeepSpeech speech-recognition engine, based on TensorFlow. The `imgViT` benchmark is a latency-critical image classification service built with Google's Vision Transformer (ViT) [18] and FastAPI. The remaining multi-threaded applications comprise a variety of HPC and scientific programs from various suites – NAS Parallel Benchmarks, Rodinia, PARSEC, and SPEC CPU (OpenMP subset)–, and a simulation benchmark based on the PBBCache cache-partitioning simulator [14].

To stress LC applications, we employ a workload generator that strives to achieve a fixed number of client requests issued per second (CRPS). To determine a suitable CRPS value for our experimental system, we followed a methodology similar to that of Chen et al. [11]: we progressively stress the LC service until it reaches a saturation point with the allocated system resources. Specifically, we ran the LC service alone on the system, gradually increasing the CRPS, while monitoring the latency distribution. Tail latency is measured at the 99th percentile for all LC applications. Maximum latency is defined as the tail latency at the critical inflection point in the CRPS-to-tail-latency curve: where a minor increase in CRPS causes a dramatic rise (several orders of magnitude) in tail latency. For multi-container workloads, we set the CRPS value just below that inflection point, which ensures that – when running in isolation – tail latency remains within the maximum threshold while approaching the lowest CRPS setting that would cause QoS violations (i.e., tail latency exceeding the maximum latency).

In running the workloads, we follow a similar approach to that of [8, 32], which focuses on keeping the system load constant throughout an experiment while factoring in the disparities in the duration of the various batch applications present in the mix.

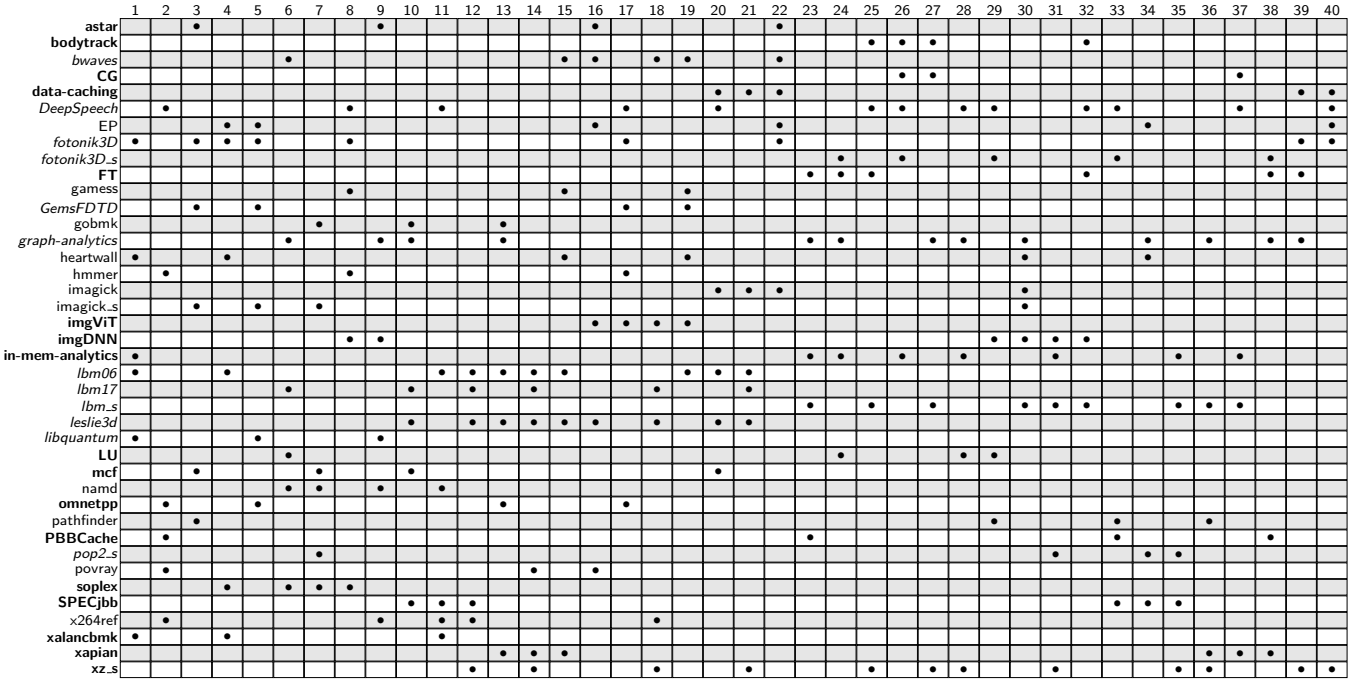


Figure 9: Workload composition. A dot in a cell indicates that the application in the cell's row number is present in the workload associated with the cell's column.

Specifically, we ensure that all applications in the workload are started simultaneously. When a batch program completes, it is restarted repeatedly until the longest batch application in the set completes at least three times and has run for at least ten minutes. When that happens, we stop the remaining applications, including the LC services. We then measure Unfairness and STP (see Sec. 2) by considering the RPS achieved by each LC service throughout the experiment, as well as the geometric mean of completion times observed for every full execution of each batch program.

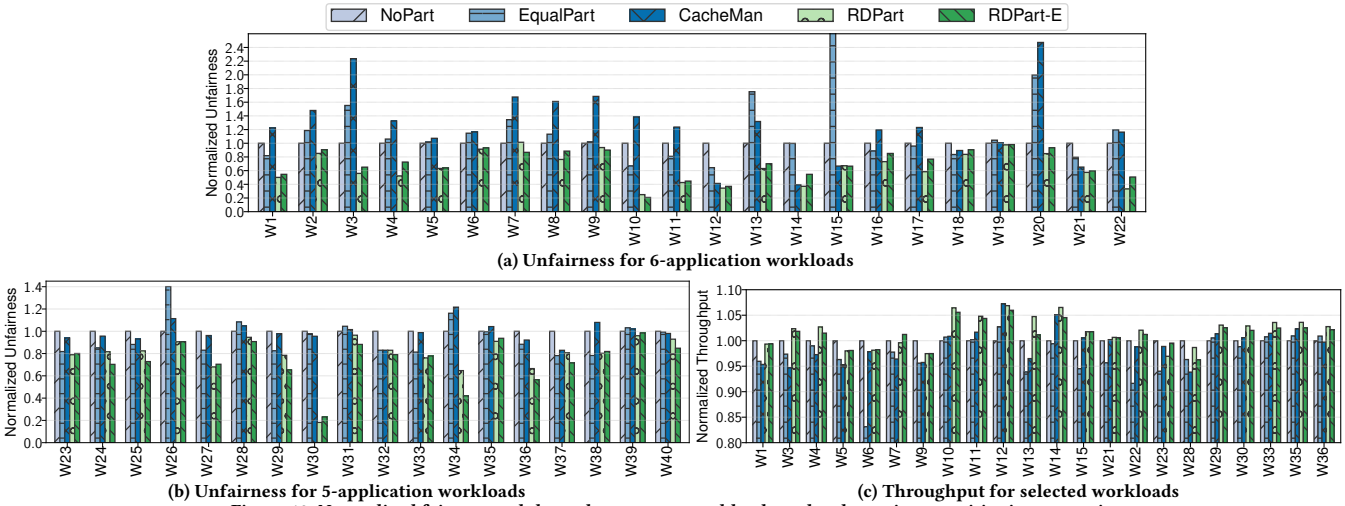
To assess the effectiveness of the two variants of RDPart, we experimentally compared it with three strategies: *NoPart*, the unmodified Linux kernel v6.2.16, which does not partition the LLC; *EqualPart*, a strategy that assigns same-sized separate partitions to each application, evenly distributing LLC space among them; and *CacheMan* [15], a state-of-the-art fairness-oriented cache-partitioning approach. We have implemented all partitioning strategies in the Linux kernel v6.2.16 using a PMCSched plugin [2]. Notably, as stated in Sec. 5, CacheMan is the only recent technique among previous proposals that shares many key design constraints with RDPart: (i) it effectively supports cloud workloads, (ii) it follows a lightweight black-box design amenable to kernel-level implementations and deployments in public clouds, and (iii) it does not employ invasive QoS-agnostic performance-monitoring techniques, based on altering per-application LLC allocations. We should also highlight that LFOC+ [29], by which RDPart is inspired, cannot be evaluated in our setting, since it does not support many cloud workloads, due to its inability to handle multi-process applications (e.g., Apache Spark based) and its reliance on a profiling mode tailored to long-running CPU-bound applications, as discussed in Sec. 3.2.

All strategies invoke their partitioning algorithm every 500ms, a usual choice in recent works [8, 13, 30]. To ensure this, RDPart's *monitoring period* is set to 500ms. At the end of this period, RDPart performs per-application classification and runs the partitioning algorithm. In our experiments, this low-overhead periodic process takes 16.4μs on average to complete on one core of the system, which is negligible compared to the monitoring period's duration.

6.2 Discussion

Figs. 10a and 10b show the unfairness values (lower-is-better) registered by the various partitioning strategies for the 6-application and 5-application workloads, respectively. The values are normalized to NoPart, which provides better average fairness figures than EqualPart. Notably, both RDPart variants substantially improve fairness w.r.t. the remaining strategies across the board, with RDPart-E providing slightly smaller average fairness improvements (in a 1.2% range). Overall, RDPart reduces average unfairness by 28.5%, 29% and 32.1% relative to NoPart, EqualPart and CacheMan, respectively.

Due to space constraints, our discussion focuses on the comparison between our approach and CacheMan – the most recent partitioning technique. The substantial differences in the results of both strategies stem from RDPart's awareness of application cache-sensitivity, factored in when distributing LLC space. CacheMan does not take this aspect into account, and allocates LLC space mainly based on the container core counts. This has a substantial impact on the results for the 6-application workloads (Fig. 10a), where CacheMan imposes a low LLC occupancy on each single-threaded application running on a single-core container. Crucially, some of these programs (e.g., `astar`, *mcf* or `omnetpp`) are highly cache sensitive, so allotting them a low LLC occupancy produces high slowdown values. This severely impacts system-wide fairness under CacheMan, sometimes degrading fairness by 2.2× or more w.r.t. the baseline (e.g., W3 and W20). For 5-application workloads (Fig. 10b), where all applications run on 4-core containers and are less cache sensitive, CacheMan delivers closer fairness figures to

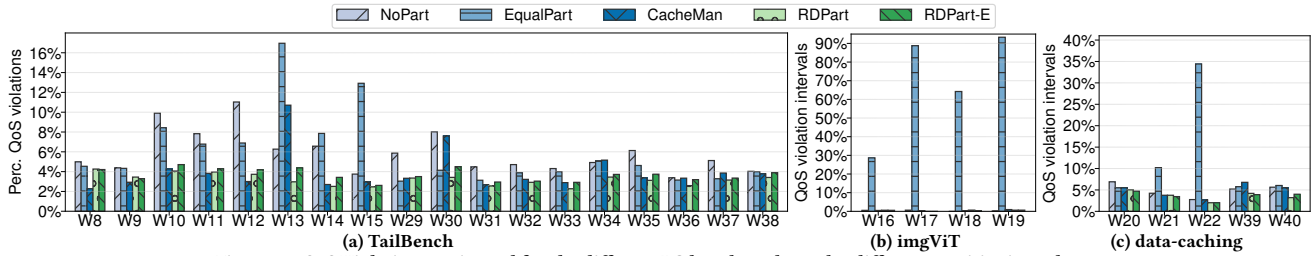


(a) Unfairness for 6-application workloads

(b) Unfairness for 5-application workloads

(c) Throughput for selected workloads

Figure 10: Normalized fairness and throughput across workloads under the various partitioning strategies



(a) TailBench (b) imgViT (c) data-caching

Figure 11: QoS Violations registered for the different LC benchmarks under different partitioning schemes

those of EqualPart, despite leveraging different methods (using overlapping vs. separate partitions) to impose similar per-application occupancies. RDPart successfully improves fairness in both workload scenarios, underscoring the crucial role of application cache sensitivity in fairness-optimized LLC partitioning.

Regarding system throughput, the differences between cache-partitioning strategies are subtle. In fact, for almost half of the workloads the throughput improvement and degradation figures associated with the various approaches are within a tight 2.5% range. For the sake of simplicity, Fig. 10c shows the normalized throughput numbers for the remaining workloads, where higher differences are observed. For these workloads, RDPart improves STP by up to 7%, and by roughly 2% on average. Notably, in those scenarios where it delivers a lower throughput than the baseline, the degradation does not exceed 3%. Therefore, we conclude that RDPart's reductions in unfairness (of up to 82% relative to NoPart) do not significantly impact system throughput.

Lastly, we turn our attention to Fig. 11, which summarizes the QoS results for workloads including latency-critical (LC) applications. All the LC services considered fall in the cache-sensitive category, and most of them are also sensitive to memory-bandwidth contention. Fig. 11a shows the percentage of QoS violations registered while running workloads that include an LC application from the Tailbench suite [17]. Clearly, NoPart and EqualPart deliver the worst results across the board, with a fraction of QoS violations exceeding 8% in some workloads. RDPart, by contrast, exhibits a consistent percentage of QoS violations (3.2% on average), and no greater than 4.2% for any workload.

CacheMan is able to match and even reduce the percentage of QoS violations relative to RDPart in some cases. However, unlike RDPart, CacheMan fails to ensure consistent fairness and QoS results across workloads. Take, for instance, the W13 and W30 workloads, where it clearly delivers worse fairness results than RDPart, while also causing a large fraction of QoS violations (over 7.8%). While CacheMan allocates sufficient space in the LLC for LC applications in this context, it fails to fulfill the LLC-space demands of other cache-sensitive applications in the workload. This triggers abundant LLC misses, leading to high memory bandwidth consumption and additional contention. RDPart, by contrast, assigns a larger LLC share to all cache-sensitive applications, which directly reduces both LLC misses and memory bandwidth consumption. The overarching conclusion is that, while RDPart does not explicitly address memory-bandwidth contention, it reduces this source of contention thanks to its LLC-partitioning method. This helps maintain QoS violations under control in this context.

The results obtained for the `imgViT` benchmark (Fig. 11b) reflect a different trend. EqualPart is the only partitioning strategy causing numerous QoS violations for this application. This stems from the fact that it assigns a small LLC partition to this application during a large fraction of the experiment, which is not sufficient to accommodate Google's ViT model [18], used for image classification. The remaining partitioning strategies allocate a larger share of the LLC, making it possible to accommodate the model in the cache, thus removing QoS violations.

The `data-caching` benchmark is the least cache-sensitive LC application among those explored in this work. QoS violations are reduced substantially when granting an LLC occupancy of over

3MB for the application. EqualPart is unable to guarantee that minimal LLC occupancy for `data-caching` during the execution of 6-application workloads (W20-W22). As shown in Fig. 11c, this causes a larger fraction of QoS violations than under the remaining strategies, which do ensure LLC occupancies greater than 3MB. Despite fulfilling the LLC-space demands, QoS violations are not completely removed, as this benchmark is sensitive to memory-bandwidth contention, which is not explicitly addressed by any of the explored techniques. Nevertheless, the reduction of LLC misses under RDPart makes it possible to achieve the lowest fraction of QoS violations among strategies in this set of workloads.

7 Conclusions and Future Work

In this work, we propose RDPart, an OS-level cache-partitioning policy that strives to improve fairness while preserving the QoS of latency-critical (LC) workloads. Our proposal follows a black-box design suitable for public cloud environments, where applications do not provide any real-time QoS feedback (e.g., tail latency) to the underlying resource manager. Unlike other black-box partitioning techniques, RDPart fully supports cloud workloads – many techniques support single-threaded applications only [7, 12, 13, 27, 30]. Moreover, it eliminates the need for online performance profiling under varying LLC allocations, which is paramount to avoid undesired QoS violations typically incurred by invasive QoS-agnostic exploratory approaches [8, 12, 13, 23, 28, 29].

Other key features of RDPart include its lightweight application-classification method based on continuous per-thread monitoring, and its reliance on cache-reuse metrics to identify applications that are more likely to suffer significantly from LLC contention. RDPart effectively assigns a large LLC share to these applications, which is crucial to minimizing system-wide unfairness and QoS violations.

We implement RDPart in the Linux kernel and evaluate its effectiveness across a diverse set of workloads, including LC cloud services, AI-based workloads and HPC/scientific applications. We experimentally compare RDPart with other partitioning strategies, including CacheMan [15], a recently proposed kernel-level black-box partitioning approach also designed to deliver fairness in public cloud settings. Results show that RDPart reduces average unfairness by 28.5% w.r.t. the default Linux scheduler that does not partition the LLC, and by 32.1% relative to CacheMan. In addition, unlike the remaining proposals, RDPart ensures a consistently small fraction of QoS violations across all explored workloads.

A promising research direction for future work is to augment RDPart with support for explicit contention management in other shared resources such as disks, network and memory bandwidth.

Acknowledgments

This work has been supported by the Spanish MCIU under Grants PID2021-126576NB-I00 and PID2024-158311NB-I00, funded also by MCIN/AEI/10.13039/501100011033 and "ERDF A way of making Europe".

References

[1] 2026. Intel Resource Director Technology (Intel RDT). https://eci.intel.com/docs/3.3/development/performance/intel-pqos.html.
[2] 2026. PMCTrack v4.0 release, including PMCSched's support for containerized applications. https://github.com/jcsaezal/pmctrack/releases/tag/v4.0.
[3] 2026. RDPart's source code. https://pmctrack-linux.github.io/rdpart.
[4] Alaa R. Alameldeen and David A. Wood. 2006. IPC Considered Harmful for Multiprocessor Workloads. *IEEE Micro* 26, 4 (2006).
[5] AMD. 2025. Processor Programming Reference (PPR) for AMD Family 1Ah Model 70h. https://docs.amd.com/v/u/en-US/57930-A0-PUB_3.00.
[6] AMD. 2026. AMD64 Zen6 Platform Quality of Service (PQOS) Extensions. https://docs.amd.com/v/u/en-US/69193_1.00_AMD64_PQOS_EXT_PUB.
[7] Yang Bai et al. 2025. PaLLOC: Pairwise-based low-latency online coordinated resource manager of last-level cache and memory bandwidth on multicore systems. *J. of Systems Architecture* 164 (2025), 103427.
[8] Carlos Bilbao et al. 2023. Divide&Content: A Fair OS-Level Resource Manager for Contention Balancing on NUMA Multicores. *IEEE Trans. Par. Distrib. Sys.* (aug 2023), 1–17.
[9] Carlos Bilbao et al. 2023. Flexible system software scheduling for asymmetric multicore systems with PMCSched: A case for Intel Alder Lake. *Concurr. Comput. Pract. Exp.* 35, 25 (2023), e7814.
[10] J. Brock et al. 2015. Optimal Cache Partition-Sharing. In *Proc. of ICPP '15'*. 749–758.
[11] Shuang Chen et al. 2019. PARTIES: QoS-Aware Resource Partitioning for Multiple Interactive Services. In *Proc. of ASPLOS '19* (Providence, RI, USA). 107–120.
[12] N. El-Sayed et al. 2018. KPart: A Hybrid Cache Partitioning-Sharing Technique for Commodity Multicores. In *Proc. of HPCA '18.* 104–117.
[13] A. Garcia-Garcia et al. 2019. LFOC: A Lightweight Fairness-Oriented Cache Clustering Policy for Commodity Multicores. In *Proc. of ICPP'19.* Article 14.
[14] A. Garcia-Garcia et al. 2020. PBBCache: an open-source parallel simulator for rapid prototyping and evaluation of cache-partitioning and cache-clustering policies. *J. Computat. Science* (2020), 101102.
[15] Xiaokang Hu et al. 2026. Cacheman: A Comprehensive Last-Level Cache Management System for Multi-tenant Clouds. In *Proc. of PPoPP '26.* 329–341.
[16] Intel. 2026. PerfMon Events Documentation. Skylake Server Events. https://perfmon-events.intel.com/platforms/skylakex/core-events/core/.
[17] Harshad Kasture and Daniel Sanchez. 2016. Tailbench: a benchmark suite and evaluation methodology for latency-critical applications. In *Proc. of IISWC'16.*
[18] Alexander Kolesnikov et al. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *ICLR Conf. '21.*
[19] D. Lo et al. 2015. Heracles: improving resource efficiency at scale. In *Proc. of ISCA '15.* 450–462.
[20] Xiaoyang Lu et al. 2024. CHROME: Concurrency-Aware Holistic Cache Management Framework with Online Reinforcement Learning. In *Proc. of HPCA'24.*
[21] S. Mittal. 2017. A Survey of Techniques for Cache Partitioning in Multicore Processors. *ACM Comput. Surv.* 50(2), Article 27 (2017), 27:1–39 pages.
[22] OpenBenchmarking. 2025. DeepSpeech benchmark. https://openbenchmarking.org/test/pts/deepspeech.
[23] Jinsu Park et al. 2019. CoPart: Coordinated Partitioning of Last-Level Cache and Memory Bandwidth for Fairness-Aware Workload Consolidation on Commodity Servers. In *Proc. of EuroSys '19.* Article 10, 16 pages.
[24] PARSA-EPFL. 2023. Cloudsuite 4.0: A Benchmark Suite for Cloud Services. https://github.com/parsa-epfl/cloudsuite.
[25] PARSA-EPFL. 2023. Data Caching Benchmark. https://github.com/parsa-epfl/cloudsuite/blob/main/docs/benchmarks/data-caching.md.
[26] T. Patel and D. Tiwari. 2020. CLITE: Efficient and QoS-Aware Co-Location of Multiple Latency-Critical Jobs for Warehouse Scale Computers. In *Proc. of HPCA '20).* 193–206.
[27] Lucia Pons et al. 2022. Cache-Poll: Containing Pollution in Non-Inclusive Caches Through Cache Partitioning. In *Proc. of ICPP' 22.* Article 33, 11 pages.
[28] R. B. Roy et al. 2021. SATORI: Efficient and Fair Resource Partitioning by Sacrificing Short-Term Benefits for Long-Term Gains. In *Proc. of ISCA '21.* 292–305.
[29] J. C. Saez et al. 2022. LFOC+: A Fair OS-level Cache-Clustering Policy for Commodity Multicore Systems. *IEEE Trans. Comp.* (2022).
[30] V. Selfa et al. 2017. Application Clustering Policies to Address System Fairness with Intel's Cache Allocation Technology. In *Proc. of PACT '17.* 194–205.
[31] Mohammad Shahrad et al. 2021. Provisioning Differentiated Last-Level Cache Allocations to VMs in Public Clouds. In *Proc. of SoCC '21.* 319–334.
[32] Daniel Shelepov et al. 2009. HASS: A Scheduler for Heterogeneous Multicore Systems. *SIGOPS Oper. Syst. Rev.* 43, 2 (2009), 66–75.
[33] Yaocheng Xiang et al. 2018. DCAPS: Dynamic Cache Allocation with Partial Sharing. In *Proc. of EuroSys '18.* Article 13.
[34] H. Yun et al. 2014. PALLOC: DRAM bank-aware memory allocator for performance isolation on multicore platforms. In *Proc. of RTAS '14.* 155–166.
[35] Laiping Zhao et al. 2024. Component-distinguishable Co-location and Resource Reclamation for High-throughput Computing. *ACM Trans. Comput. Syst.* 42, 1–2, Article 2 (feb 2024), 37 pages.
[36] Yuhong Zhong et al. 2024. Managing Memory Tiers with CXL in Virtualized Environments. In *Proc. of OSDI'24.* 37–56.